1. Constructor
    - The purpose of the ctor is to initialize the state of an object

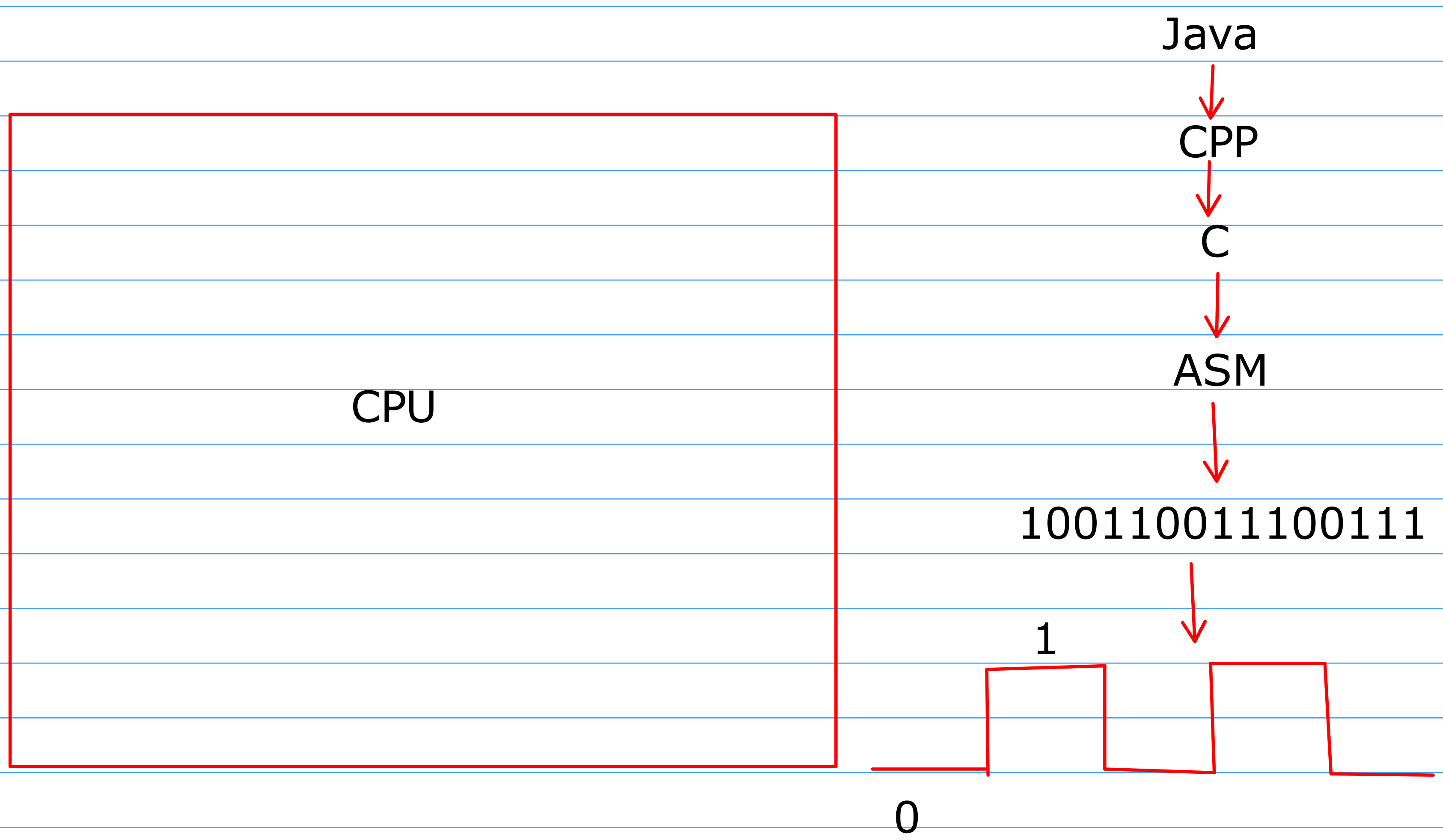
Types of Constructors
1. Defualt/Paramaterless ctor
2. Parameterized ctor

Default Ctor
- It is a ctor which is paramaterless ctor
- It is added by the compiler if no ctor exists inside your class
- It allows us to create the object without passing any arguments
- If any user defined ctor exists inside the class then default ctor is not added in the class

2. Setter / Mutators

3. Getter / Inspectors

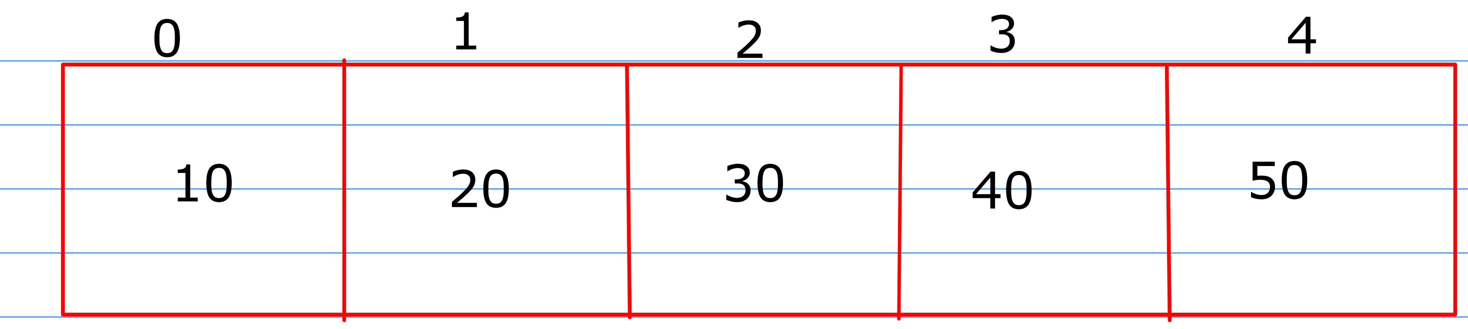
4. Facilitator



Ctor chaining
- It is way through which we can reuse the body of a paramaterized ctor
- to perform ctor chaining we have to use this() statement
- this statement must be the first statement inside the ctor

Array
- It is a DataStructure used to store the elements of similar type in contigious memory location
- It is having a fixed size
- It provides the index based operation

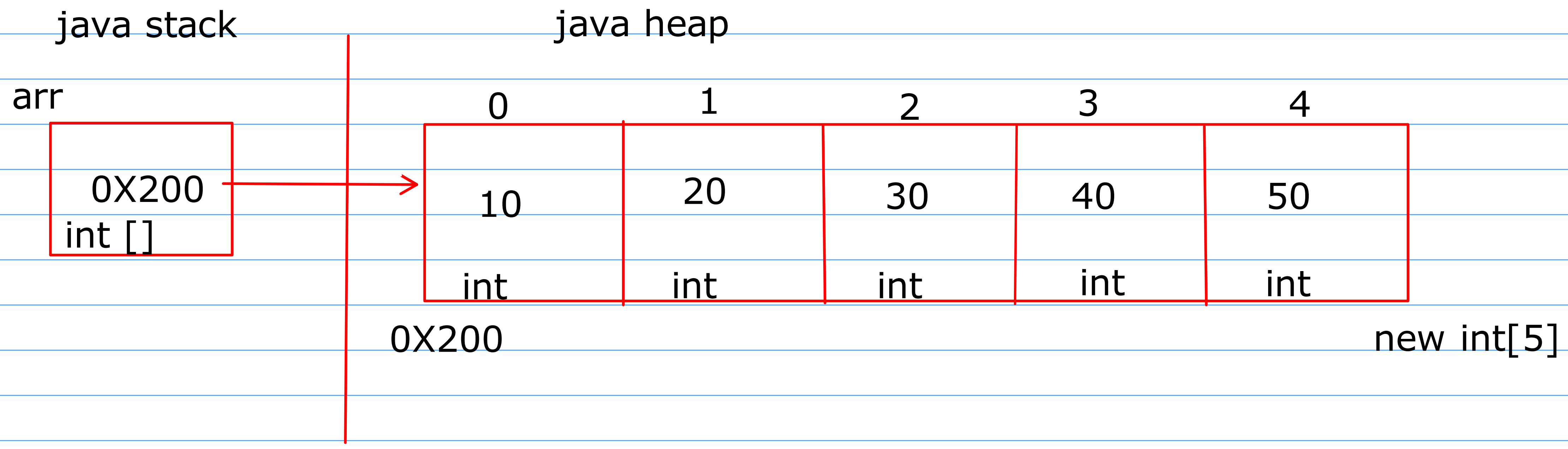
int arr[5];



0X200
arr

```
int val=arr[0]
arr[0] = 10
```

```
for(int i = 0; i<5;i++){
        print(arr[i])
}
```



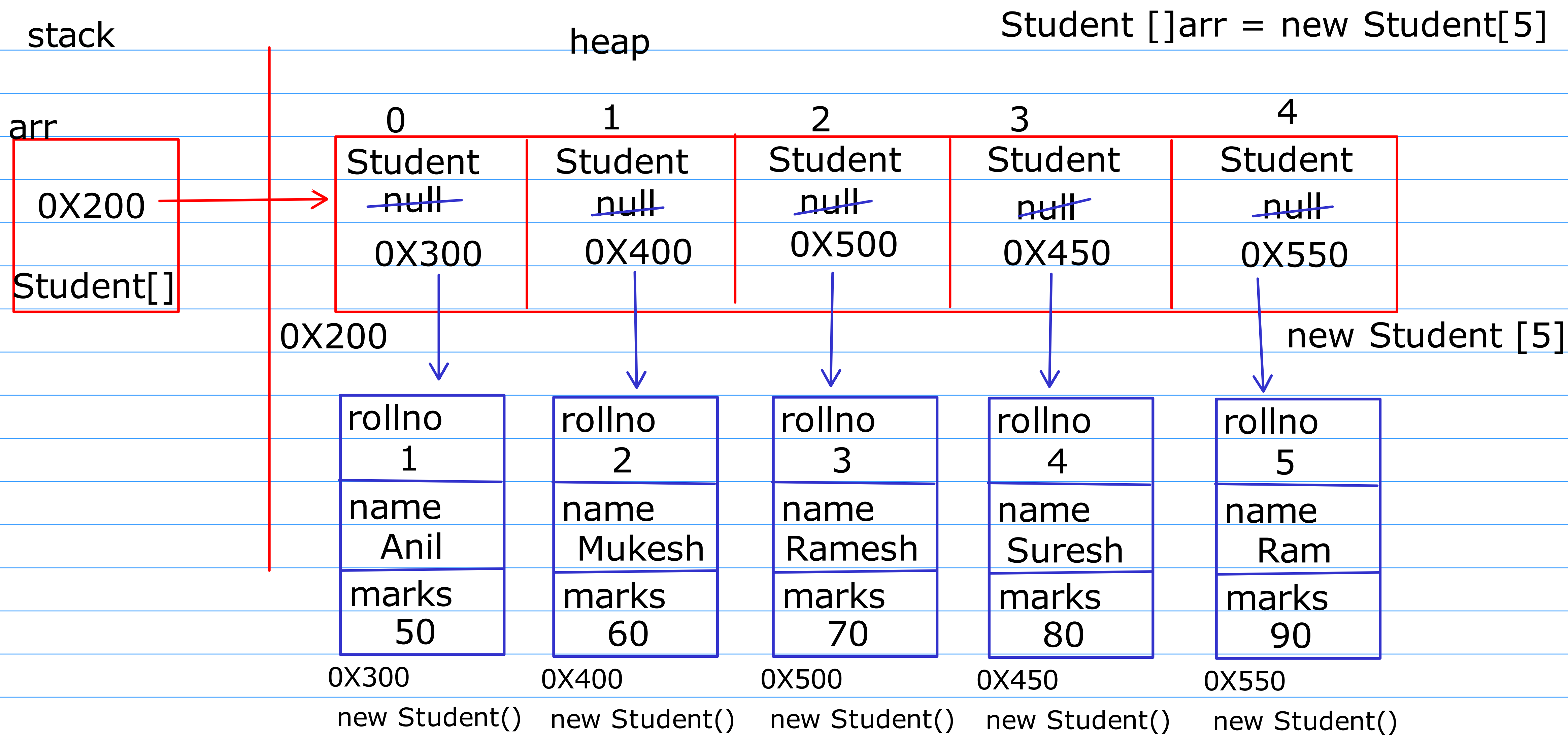

```
int[] arr = new int[5];
```

```
0X200[0] = 10;
arr[0] = 10;
arr[1] = 20;
```

```
for(int e:arr)
```

```
int arr[5];
int *ptr = new int[5]; // cpp
int *ptr = (int *) malloc(sizeof(int)*5);//c
```

Types of Array
1. Single Dimensional Array -> Primitive type, Non primitive
2. MultiDimensional Array
3. Ragged Array

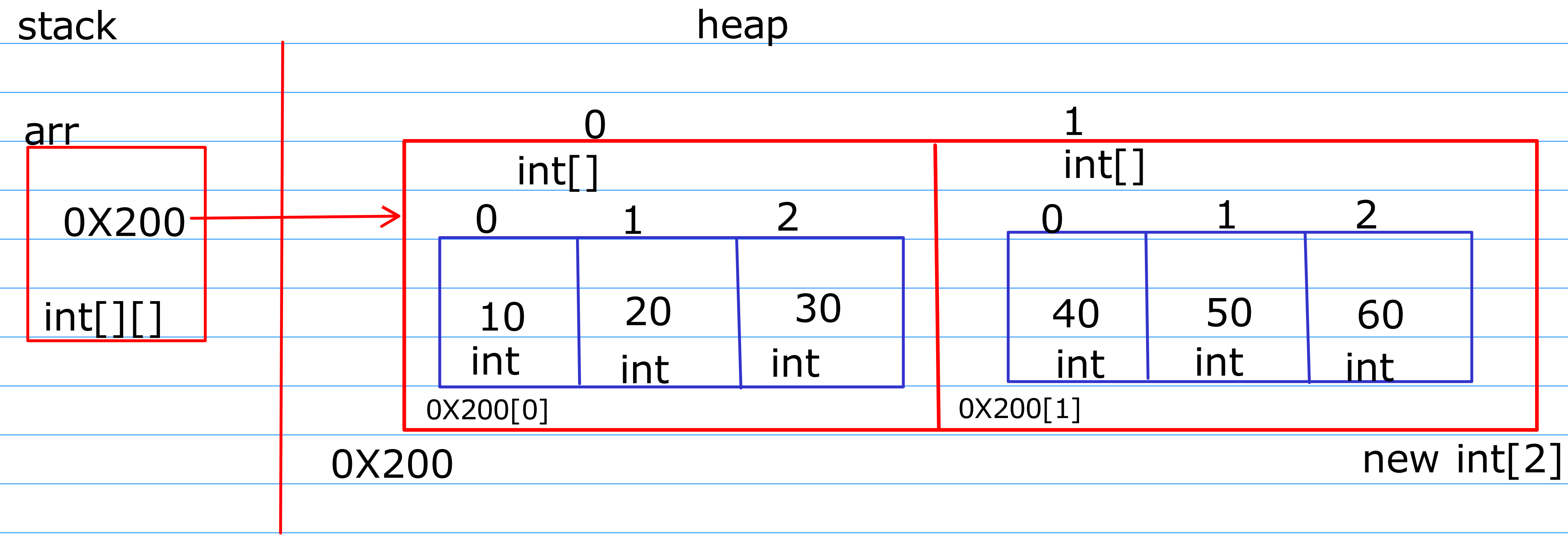



```
Student[] arr = new Student[5];
arr[0] = new Student(1, "Anil", 50);
arr[1] = new Student(2, "Mukesh", 60);
arr[2] = new Student(3, "Ramesh", 70);
arr[3] = new Student(4, "Suresh", 80);
arr[4] = new Student(5, "Ram", 90);
```

Multidimensional Array

Prmitive type

int[][] arr = new int[2][3]



0X200[0][0] = 10
arr[0][0]

0X200[1][0] = 40
arr[1][0] = 40

```
for(int i = 0;i<arr.length;i++){
        for(int j=0;j<arr[i].length;j++){
        print(arr[i][j])
}
}
```

```
for(int[] e:arr){
        for(int n:e)
                print(n)
}
```

0X300[0] = 10
0X200[0][0] = 10